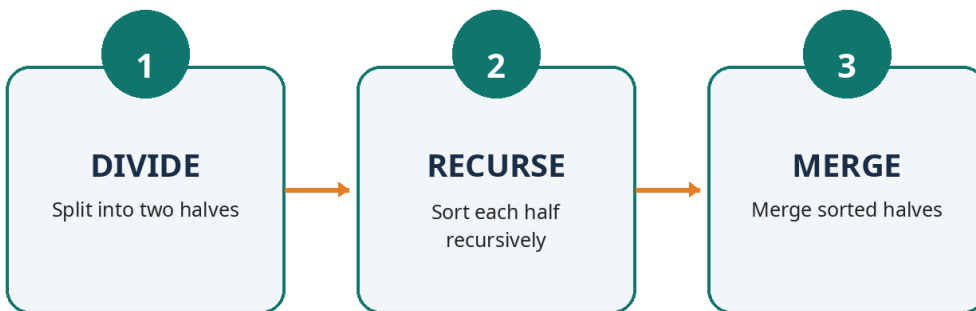


MERGE SORT

การเรียงลำดับแบบผสาน

Merge Sort Workflow



Core idea: split smaller -> sort subproblems -> merge in order

ภาพหน้าปก: แนวคิดหลักของ Merge Sort - Divide, Recurse และ Merge

หนังสือเนื้อหาเฉพาะเรื่อง Merge Sort สำหรับวิชา Algorithm

ออกแบบให้ใช้ได้ทั้งประกอบการสอน การทบทวนด้วยตนเอง และเป็นฐานความรู้สำหรับระบบ RAG

ขอบเขตของเล่ม เริ่มที่ Merge Sort โดยตรง ไม่อธิบายความหมายทั่วไปของ Algorithm และเน้นให้ผู้อ่านเข้าใจทั้งแนวคิด ขั้นตอน โค้ด และการวิเคราะห์ประสิทธิภาพ

ก่อนเริ่ม: หนังสือเล่มนี้สอนอะไร

Merge Sort เป็นวิธีเรียงลำดับที่อาศัยแนวคิดแบ่งปัญหออกเป็นส่วนย่อย จากนั้นเรียงแต่ละส่วนและนำกลับมารวมกันอย่างเป็นลำดับ จุดที่ทำให้ Merge Sort แตกต่างจากการเรียงแบบง่ายหลายวิธีคือ ขั้นตอน Merge สามารถรวมข้อมูลที่ “เรียงแล้ว” สองชุดได้ในเวลาเชิงเส้น

เมื่อเรียนจบเล่มนี้ ผู้อ่านควรทำได้

- อธิบายลำดับ Divide & Recursive Sort Merge ได้ด้วยภาษาของตนเอง
- วาดการแบ่ง Array จนถึง Base Case และรวมกลับได้ถูกต้อง
- Dry run ขั้นตอน Merge โดยใช้ตัวชี้ i และ j ได้
- เขียน Pseudocode และ Python สำหรับ Merge Sort ได้
- อธิบายเหตุผลของเวลา $\Theta(n \log n)$ และพื้นที่เสริม $O(n)$ สำหรับ Array implementation ทัวไปได้
- อธิบาย Stability, Base Case, Recursion, Bottom-up Merge Sort, Linked-list Merge Sort และ External Merge Sort ได้

สำหรับ **RAG** หัวข้อสำคัญจะเขียนให้มีความหมายในตัวเอง มีคำศัพท์ไทย-อังกฤษ และมีคำอธิบายได้ภาพ เพื่อให้ระบบ Retrieval ยังตอบได้แม้ไม่ได้ใช้ความสามารถด้าน Vision

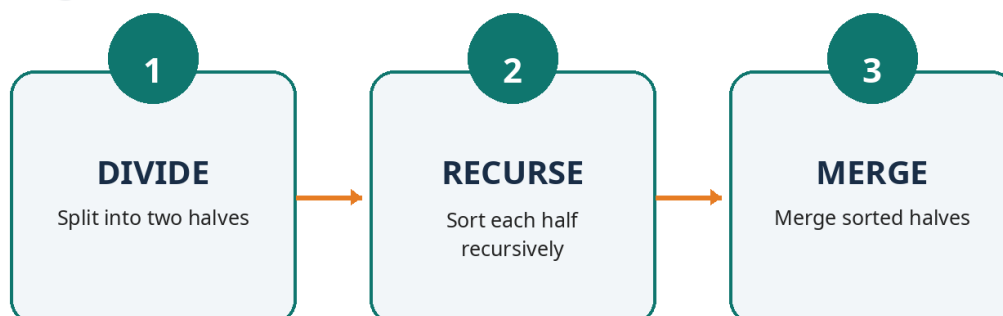
สารบัญ

1. ภาพรวมของ Merge Sort
2. Divide and Conquer ใน Merge Sort
3. ขั้นตอน Divide และการหาจุดกึ่งกลาง
4. Recursion และ Base Case
5. ขั้นตอน Merge: หัวใจของ Merge Sort
6. Dry Run แบบเต็ม
7. Pseudocode
8. Python Implementation
9. การวิเคราะห์ Time Complexity
10. Space Complexity และหน่วยความจำ
11. Stability และคุณสมบัติสำคัญ
12. Bottom-up Merge Sort
13. Merge Sort กับ Linked List
14. External Merge Sort
15. ข้อผิดพลาดที่พบบ่อย
16. คำถามที่มักสับสน (FAQ)
17. สรุปแนวคิด
18. แบบฝึกหัดและเฉลย
19. Glossary สำหรับการเรียนและ RAG
20. เอกสารอ้างอิงและสื่อภาพ

1. ภาพรวมของ Merge Sort

Merge Sort คือการเรียงลำดับแบบเปรียบเทียบ (comparison sorting) ที่แบ่งข้อมูลเป็นสองส่วน เรียงแต่ละส่วนด้วยวิธีเดิม และรวมสองส่วนที่เรียงแล้วเข้าด้วยกัน กระบวนการนี้เกิดซ้ำจนได้ข้อมูลที่เรียงครบทั้งหมด

Merge Sort Workflow



Core idea: split smaller -> sort subproblems -> merge in order

ภาพที่ 1.1 ลำดับความคิดหลักของ Merge Sort: Divide → Recurse → Merge

ภาพจำที่ควรจำให้ได้

สูตรจำ แบ่งให้เล็กที่สุด → กลับขึ้นมาทีละระดับ → ทุกครั้งที่กลับขึ้นมาให้ Merge สองชุดที่เรียงแล้ว

หากต้องเรียง 8 3 5 1 ขั้นแรกยังไม่พยายามสลับตำแหน่งที่ละคู่ แต่แบ่งเป็น [8, 3] และ [5, 1] ก่อน จากนั้นแต่ละส่วนถูกแบ่งต่อจนเหลือสมาชิกหนึ่งตัว แล้วจึงรวมกลับเป็น [3, 8] และ [1, 5] ก่อนรวมครั้งสุดท้ายเป็น [1, 3, 5, 8]

2. Divide and Conquer ใน Merge Sort

2.1 Divide - แบ่งปัญหา

แบ่ง Array ออกเป็น Left และ Right โดยให้ขนาดใกล้เคียงกันมากที่สุด หากจำนวนสมาชิกเป็นเลขคี่ ความยาวสองฝั่งต่างกันได้หนึ่งตำแหน่ง ซึ่งไม่เป็นปัญหา

2.2 Conquer - แก้ปัญหาย่อย

เรียก Merge Sort กับ Left และ Right อีกครั้ง การเรียกตัวเองนี้คือ Recursion และจะทำต่อจนถึง Base Case

2.3 Combine - รวมคำตอบ

เมื่อ Left และ Right เรียงแล้ว จะใช้ขั้น Merge เลือกค่าที่น้อยกว่าจากด้านหน้าของแต่ละฝั่งไปใส่ผลลัพธ์ทีละตัว จนได้ชุดใหม่ที่เรียงแล้ว

ช่วง	คำถามที่ควรถามตัวเอง	ผลที่ต้องได้
Divide	จะแบ่งตรงไหน?	สอง subarray ขนาดใกล้เคียงกัน
Conquer	เมื่อไรหยุดเรียกตัวเอง?	ความยาว ≤ 1
Combine	จะรวมอย่างไรให้ยังเรียงอยู่?	เปรียบเทียบค่าด้านหน้าของสองฝั่ง

สำคัญ คำว่า “Merge” ไม่ได้หมายถึงเอาสอง Array มาต่อกันเฉย ๆ แต่หมายถึง “รวมสองชุดที่เรียงแล้วโดยยังรักษาลำดับ”

3. ขั้น Divide และการหาจุดกึ่งกลาง

สำหรับ Array ความยาว n นิยมเลือกจุดกึ่งกลางด้วย integer division เช่น $\text{mid} = \text{len}(\text{arr}) // 2$ แล้วแบ่งเป็น $\text{arr}[:\text{mid}]$ และ $\text{arr}[\text{mid}:]$

```
arr = [38, 27, 43, 3, 9, 82, 10]
mid = len(arr) // 2      # 7 // 2 = 3
left = arr[:mid]         # [38, 27, 43]
right = arr[mid:]        # [3, 9, 82, 10]
```

ทำไมจำนวนคี่จึงไม่เป็นปัญหา

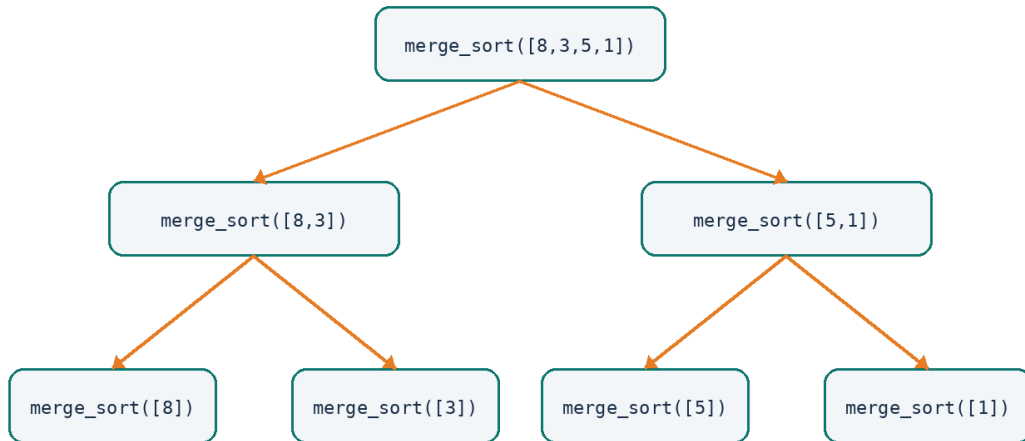
Merge Sort ไม่ได้ต้องการให้สองฝั่งมีขนาดเท่ากันเป๊ะ ขอเพียงแบ่งให้ลดขนาดลงอย่างต่อเนื่อง ในตัวอย่าง 7 ตัว ฝั่งหนึ่งมี 3 ตัวและอีกฝั่งมี 4 ตัว ทั้งสองฝั่งยังเล็กลงและไปถึง Base Case ได้

ข้อควรระวัง

- จุดแบ่งต้องทำให้ปัญหาย่อยเล็กลงจริง ไม่เช่นนั้น recursion อาจไม่จบ
- ภาษาแต่ละภาษาอาจจัดการ slice ต่างกัน บางภาษา slice สร้างสำเนาและมีต้นทุนเพิ่ม
- สามารถเขียนแบบส่ง index แทนการสร้าง subarray เพื่อหลีกเลี่ยงต้นทุนของ slice ได้

4. Recursion และ Base Case

Recursive Calls and Base Case



Base case: stop when length ≤ 1 ; 0 or 1 item is already sorted

Then return upward and merge from small sublists to larger ones

ภาพที่ 4.1 การเรียก Merge Sort ซ้ำกับครึ่งซ้ายและครึ่งขวา จนถึง Base Case

Base Case ของ Merge Sort โดยทั่วไปคือข้อมูลที่มีสมาชิกไม่เกินหนึ่งตัว เพราะชุดดังกล่าวถือว่าเรียงอยู่แล้ว จึงไม่ต้องแบ่งต่อ

```
if len(arr) <= 1:
    return arr
```

ลำดับที่เกิดขึ้นจริง

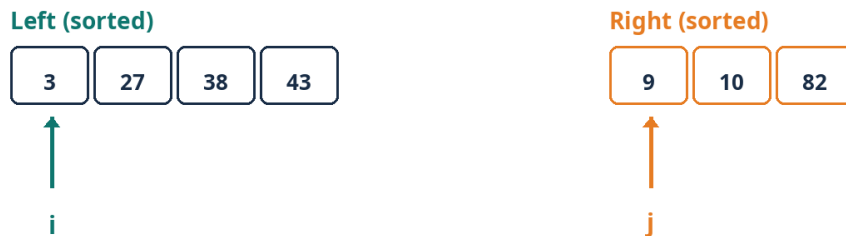
1. ฟังก์ชันรับ Array
2. ถ้ามีมากกว่า 1 ตัว ให้แบ่ง Left / Right
3. เรียก merge_sort(left)
4. เรียก merge_sort(right)
5. เมื่อทั้งสองฝั่งกลับมาเป็น sorted array แล้ว จึงเรียก merge(left, right)

จุดที่มักเข้าใจผิด Merge ไม่ได้ทำพร้อมกับการแบ่งลง แต่เกิดตอนฟังก์ชันย่อยเริ่ม return กลับขึ้นมา

5. ขั้น Merge: หัวใจของ Merge Sort

สมมติว่าเรามีข้อมูลที่เรียงแล้วสองชุด Left และ Right การ Merge ใช้ตัวชี้สองตัว i และ j ซึ่งสมาชิกที่ยังไม่ถูกนำออกของแต่ละฝั่ง เปรียบเทียบ $\text{Left}[i]$ กับ $\text{Right}[j]$ แล้วเลือกค่าน้อยกว่าใส่ผลลัพธ์

Merge Step: Choose the Smaller Front Item



Compare 3 and 9 -> choose 3 -> append to result

Move i to 27, then compare 27 with 9

Repeat until one side is empty, then append the remainder

ภาพที่ 5.1 ตัวชี้ i และ j ในขั้น Merge

กติกา Merge

1. เปรียบเทียบ $\text{Left}[i]$ กับ $\text{Right}[j]$
2. นำค่าน้อยกว่าลง Result
3. เลื่อน pointer เฉพาะฝั่งที่ถูกเลือก
4. ทำซ้ำตรรกะใดที่ทั้งสองฝั่งยังมีสมาชิก
5. เมื่อฝั่งหนึ่งหมด ให้นำสมาชิกที่เหลือของอีกฝั่งมาต่อท้ายทั้งหมด

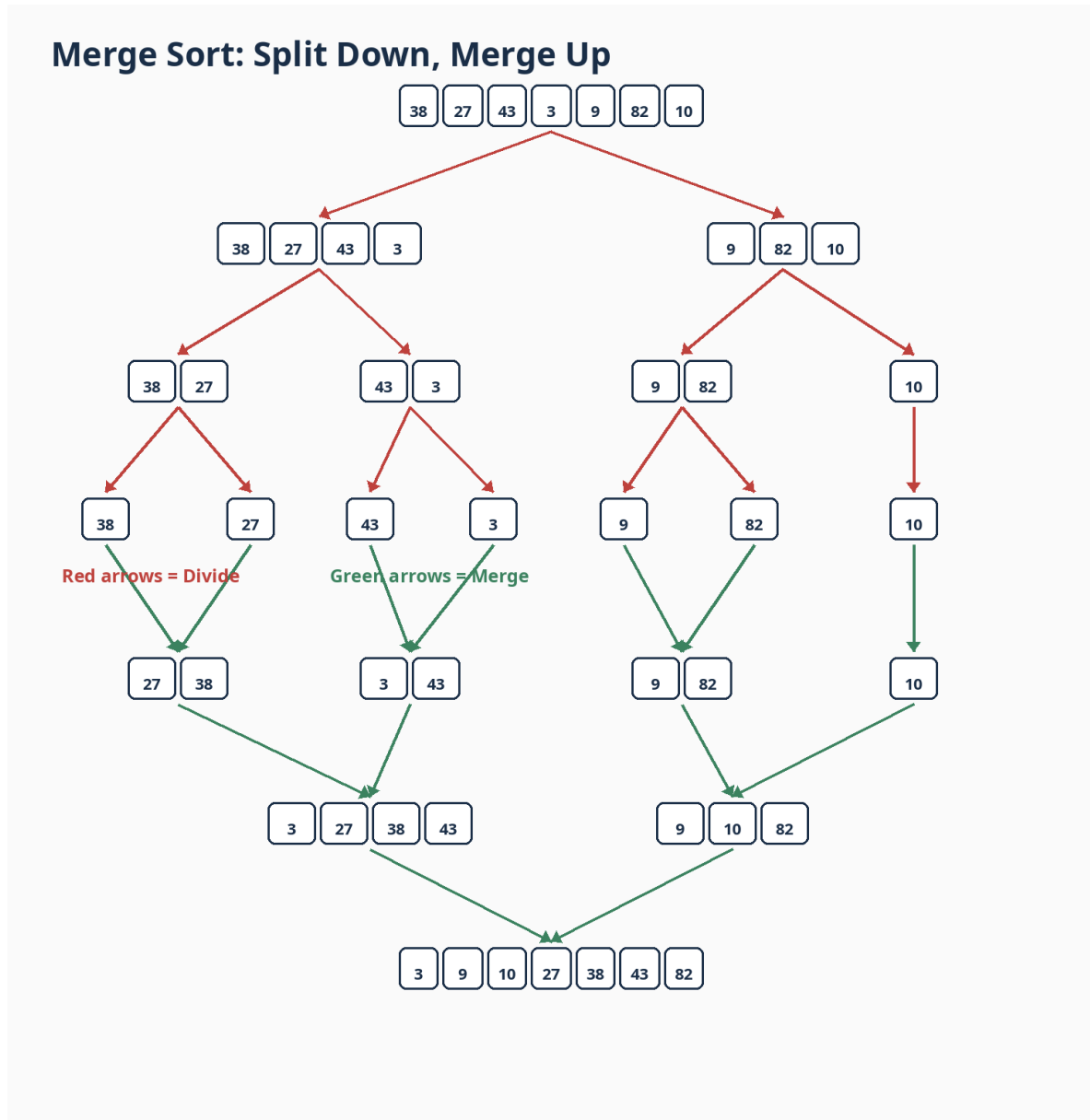
5.1 Dry Run ของ Merge ที่ละเอียด

ต้อง Merge Left = [3, 27, 38, 43] กับ Right = [9, 10, 82]

รอบ	เปรียบเทียบ	เลือก	Result หลังรอบ
1	3 กับ 9	3 จาก Left	3
2	2 7 กับ 9	9 จาก Right	[3, 9]
3	2 7 กับ 10	10 จาก Right	[3, 9, 10]
4	2 7 กับ 8 2	2 7 จาก Left	[3, 9, 10, 27]
5	3 8 กับ 8 2	3 8 จาก Left	[3, 9, 10, 27, 38]
6	4 3 กับ 8 2	4 3 จาก Left	[3, 9, 10, 27, 38, 43]
จบ	Left หมด	คัดลอก 8 2	[3, 9, 10, 27, 38, 43, 82]

สังเกต เราไม่ต้องย้อนกลับไปค้นหาค่าน้อยที่สุดจากทั้ง Array เพราะแต่ละฝั่งเรียงอยู่แล้ว ค่าที่เล็กที่สุดที่ยังเหลือจึงต้องอยู่ตรงตำแหน่ง i หรือ j

6. Dry Run แบบเต็ม: จาก Array เดิมจนเรียงเสร็จ



ภาพที่ 6.1 ขั้นตอนและรวมของ Merge Sort ใช้ชุดข้อมูล $[38, 27, 43, 3, 9, 82, 10]$ ดัดแปลงจากแผนภาพ Public Domain ของ VineetKumar บน Wikimedia Commons

อ่านภาพจากบนลงล่างในช่วงลูกศรแดงเพื่อดูการ Divide และอ่านจากกลางลงล่างในช่วงลูกศรเขียวเพื่อดูการ Merge จากชุดสมาชิกหนึ่งตัวกลับเป็นชุดที่ใหญ่ขึ้น

ฝึกอ่านภาพ ลองปิดแถวล่างสุด แล้วทำนายว่าการ Merge $[3, 27, 38, 43]$ กับ $[9, 10, 82]$ จะให้ผลอะไร ก่อนเปิดดูคำตอบ

6.1 กรณีพิเศษที่ควร Dry Run ให้เป็น

กรณีจำนวนสมาชิกเป็นเลขคี่

ตัวอย่าง [7, 2, 9, 1, 5] สามารถแบ่งเป็น [7, 2] และ [9, 1, 5] หรืออีกนโยบายหนึ่งที่ทำให้ฝั่งซ้ายใหญ่กว่าได้ สิ่งสำคัญคือทุก recursion ต้องลดขนาดและสุดท้ายถึง Base Case

กรณีมีค่าซ้ำ

ตัวอย่าง [5, 2, 5, 1] จะได้ [1, 2, 5, 5] เช่นเดิม แต่ต้องการให้ Stable เมื่อค่าซ้ายและขวาเท่ากันควรเลือกสมาชิกจาก Left ก่อน

กรณีข้อมูลเรียงอยู่แล้ว

Merge Sort แบบมาตรฐานยังคงแบ่งและ Merge ตามโครงสร้างเดิม ดังนั้นเวลาเชิงกำกับยังเป็น $\Theta(n \log n)$ แม้ Input จะเรียงอยู่แล้ว

กรณีเรียงย้อนกลับ

เช่น [5, 4, 3, 2, 1] ชั้น Divide ยังเหมือนเดิม และ Merge จะสร้างลำดับใหม่จากค่าที่น้อยกว่าในแต่ละฝั่ง

6.2 ค่าซ้ำและ Stability

Stable Sort: Equal Keys Keep Their Original Order

Before sorting



After ascending stable sort



ภาพที่ 6.2 ตัวอย่าง Stable Sort เมื่อข้อมูลหลายรายการมี key เท่ากัน

Stable Sort หมายถึง เมื่อข้อมูลสองรายการมี key เท่ากัน ลำดับสัมพัทธ์เดิมจะยังคงอยู่หลังเรียง ตัวอย่าง B มาก่อน D ที่คะแนน 70 และ A มาก่อน C ที่คะแนน 80 ดังนั้นหลังเรียงแบบ stable ลำดับภายในกลุ่มคะแนนเดียวกันไม่สลับกัน

```
if left[i] <= right[j]:
    result.append(left[i]) # เลือกฝั่งซ้ายก่อนเมื่อค่าเท่ากัน
    i += 1
else:
    result.append(right[j])
    j += 1
```

ข้อสรุป Merge Sort สามารถทำให้ Stable ได้โดยธรรมชาติของขั้น Merge หากกำหนดกติกา tie ให้เลือกฝั่งซ้ายก่อน

7. Pseudocode

7.1 MERGE-SORT

```
MERGE-SORT(A)
  if length(A) <= 1
    return A

  mid = floor(length(A) / 2)
  left = MERGE-SORT(A[0 : mid])
  right = MERGE-SORT(A[mid : end])

  return MERGE(left, right)
```

7.2 MERGE

```
MERGE(left, right)
  result = empty list
  i = 0, j = 0

  while i < length(left) and j < length(right)
    if left[i] <= right[j]
      append left[i] to result
      i = i + 1
    else
      append right[j] to result
      j = j + 1

  append all remaining items in left
  append all remaining items in right
  return result
```

อ่าน Pseudocode ให้เชื่อมกับภาพ MERGE-SORT ทำหน้าที่ “แบ่งและเรียกซ้ำ” ส่วน MERGE ทำหน้าที่ “รวมสองชุดที่เรียงแล้ว” อย่าสับสนสองหน้าที่นี้

8. Python Implementation แบบอ่านง่าย

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

ทดลองใช้งาน

```
data = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(data))
# [3, 9, 10, 27, 38, 43, 82]
```

8.1 อธิบายโค้ดที่ละส่วน

ส่วนของโค้ด	หน้าที่
<code>if len(arr) <= 1</code>	Base Case: หยุดแบ่ง
<code>mid = (len(arr) + 1) // 2</code>	หาจุดกึ่งกลาง
<code>merge_sort(arr[:mid])</code>	เรียงครึ่งซ้ายด้วย recursion
<code>merge_sort(arr[mid:])</code>	เรียงครึ่งขวาด้วย recursion
<code>while i < ... and j < ...</code>	เปรียบเทียบตราบดที่สองฝั่งยังไม่หมด
<code>left[i] <= right[j]</code>	เลือกค่าน้อยกว่า และช่วยรักษา stability เมื่อค่าเท่ากัน
<code>result.extend(...)</code>	คัดลอกสมาชิกที่ยังเหลือ
<code>return merge(left, right)</code>	ส่ง sorted array กลับไปยัง call ก่อนหน้า

เวอร์ชันนี้เปลี่ยน Array เดิมหรือไม่?

โค้ดตัวอย่างนี้สร้าง list ใหม่และ return ผลลัพธ์ ดังนั้น data เดิมไม่ถูกจัดเรียงในตำแหน่งเดิม หากต้องการ in-place interface สามารถคัดลอกผลลัพธ์กลับไปยัง Array เดิมได้ แต่ขั้น Merge สำหรับ Array ทั่วไปยังมักใช้อาร์เรย์ช่วย

8.2 Trace Execution: [8, 3, 5, 1]

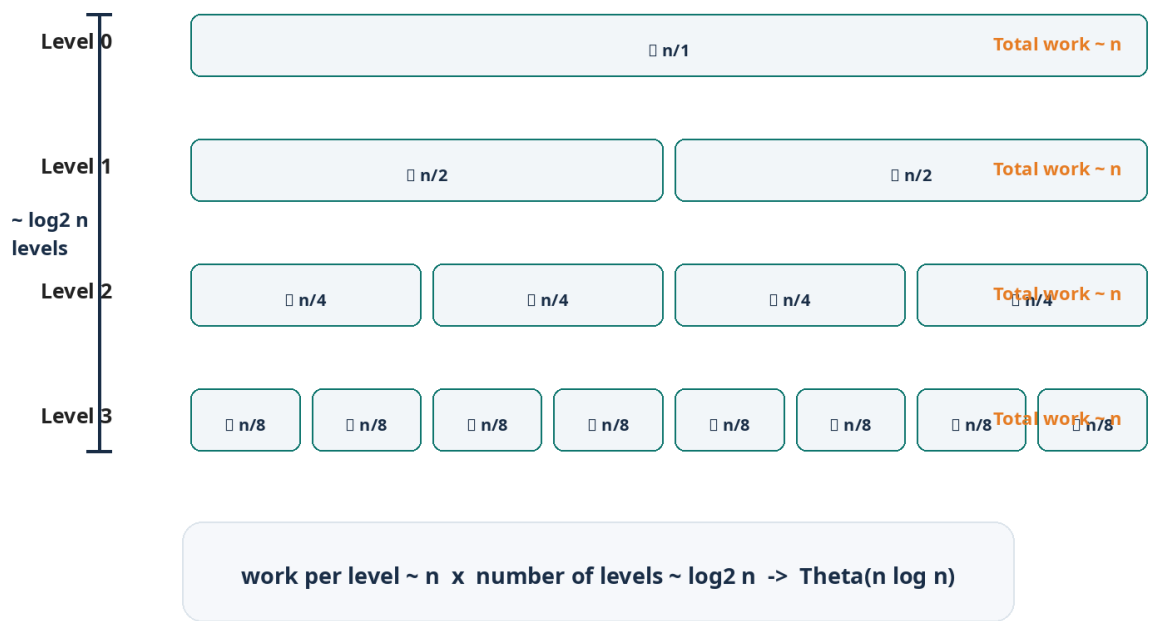
```
merge_sort([8, 3, 5, 1])
├─ merge_sort([8, 3])
│   ├─ merge_sort([8]) -> [8]
│   └─ merge_sort([3]) -> [3]
│       └─ merge([8], [3]) -> [3, 8]
├─ merge_sort([5, 1])
│   ├─ merge_sort([5]) -> [5]
│   └─ merge_sort([1]) -> [1]
│       └─ merge([5], [1]) -> [1, 5]
└─ merge([3, 8], [1, 5]) -> [1, 3, 5, 8]
```

Trace นี้แสดงชัดว่าการ Merge จะเริ่มจากล่างขึ้นบน หลังจาก recursion ไปถึง Array ขนาด 1 แล้ว

เทคนิคเวลา Debug พิมพ์ค่าก่อนแบ่งและก่อน Merge จะช่วยให้เห็น call stack และลำดับการคืนค่าได้ง่ายมาก

9. การวิเคราะห์ Time Complexity

Why the Running Time is $\Theta(n \log n)$



ภาพที่ 9.1 ในแต่ละระดับมีงานรวมประมาณ n และจำนวนระดับประมาณ $\log_2 n$

การแบ่งครึ่งทำให้ความลึกของ recursion โดดตาม $\log_2 n$ เพราะทุกระดับขนาดปัญหาจะลดลงประมาณครึ่งหนึ่ง ส่วนการ Merge ในแต่ละระดับต้องแต่ละสมาชิกรวมประมาณ n ตัว

สรุปเชิงกำกับ

กรณี	Time Complexity	เหตุผล
Best	$\Theta(n \log n)$	เวอร์ชันมาตรฐานยังแบ่งและ Merge ตามทุกระดับ
Average	$\Theta(n \log n)$	งานรวมแต่ละระดับประมาณ n
Worst	$\Theta(n \log n)$	โครงสร้างการแบ่งยังมีประมาณ $\log n$ ระดับ

จำให้แม่น $\Theta(n \log n)$ ไม่ได้มาจาก “แบ่ง = $\log n$ ” อย่างเดียว แต่เกิดจาก “ประมาณ $\log n$ ระดับ $\times$ งาน Merge รวมประมาณ n ต่อระดับ”

9.1 Recurrence และ Recursion Tree

เวลาของ Merge Sort สามารถเขียนเป็น recurrence สำหรับ n ที่มากกว่า 1 ได้ว่า

$$T(n) = 2T(n/2) + \Theta(n)$$

ส่วน $2T(n/2)$ มาจากการเรียงครึ่งซ้ายและครึ่งขวา ส่วน $\Theta(n)$ มาจากการ Merge สองครึ่งกลับเข้าด้วยกัน

มองแบบระดับ

- ระดับบนสุด: Merge ข้อมูลรวม n ตัว
- ระดับถัดไป: มี 2 งาน ขนาดประมาณ $n/2$ แต่รวมกันยังประมาณ n
- ระดับถัดไป: มี 4 งาน ขนาดประมาณ $n/4$ รวมกันยังประมาณ n
- จำนวนระดับก่อนถึงข้อมูลขนาด 1 อยู่ในระดับ $\log_2 n$

ดังนั้นผลรวมทุกระดับจึงเป็น $\Theta(n) \times \Theta(\log n) = \Theta(n \log n)$

หมายเหตุเรื่อง slicing ใน Python

การ slice เช่น `arr[:mid]` สร้าง list ใหม่และมีต้นทุนตามจำนวนสมาชิกที่คัดลอก แต่ภาพรวมของ implementation มาตรฐานยังคงอยู่ใน $\Theta(n \log n)$ เพียงมีค่า constant และการใช้หน่วยความจำเพิ่มขึ้น

10. Space Complexity และหน่วยความจำ

Array-based Merge Sort แบบมาตรฐานต้องมีพื้นที่ช่วยระหว่าง Merge เพื่อเก็บข้อมูลที่กำลังรวม ขนาดรวมสูงสุดอยู่ในระดับ $O(n)$ นอกจากนี้ recursive version มี call stack ลึกประมาณ $O(\log n)$

องค์ประกอบ	ขนาดโดยประมาณ
พื้นที่ช่วยสำหรับ Merge	$O(n)$
Call stack ของ recursion	$O(\log n)$
พื้นที่รวมเชิงกำกับ	$O(n)$ เนื่องจาก $O(n)$ ครอบ $O(\log n)$

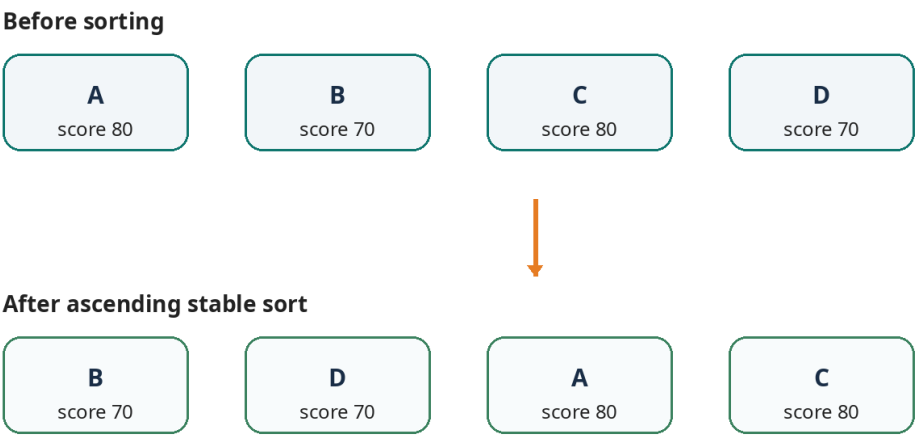
Merge Sort เป็น In-place หรือไม่?

สำหรับ Array implementation ที่ใช้กันทั่วไป ไม่ถือว่าเป็น in-place เพราะต้องใช้ auxiliary storage เพิ่ม แม้จะมีงานวิจัยและเทคนิค in-place merge ที่ซับซ้อนกว่า แต่ไม่ใช่ implementation พื้นฐานที่สอน

อย่าสับสน Time Complexity และ Space Complexity เป็นคนละมิติ: Merge Sort ใช้เวลา $\Theta(n \log n)$ แต่พื้นที่เสริมโดยทั่วไป $O(n)$

11. คุณสมบัติสำคัญของ Merge Sort

Stable Sort: Equal Keys Keep Their Original Order

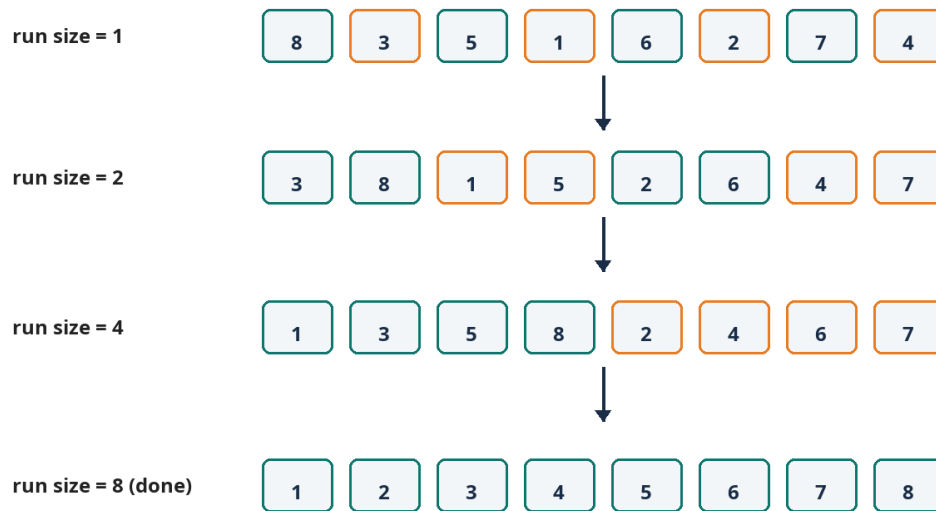


ภาพที่ 11.1 Stability เป็นคุณสมบัติที่สำคัญเมื่อ record มี key เท่ากัน

คุณสมบัติ	สถานะ/คำอธิบาย
Comparison sort	ใช่ - ตัดสินลำดับจากการเปรียบเทียบ key
Divide and conquer	ใช่ - แบ่งเป็นปัญหาย่อยแล้วรวมคำตอบ
Stable	ทำได้ และ implementation มาตรฐานมักทำให้ stable
Worst-case time	$\Theta(n \log n)$
Array auxiliary space	$O(n)$ โดยทั่วไป
เหมาะกับ Linked List	เหมาะ เพราะ Merge ทำได้ด้วยการเชื่อม node
เหมาะกับข้อมูลใหญ่กว่าหน่วยความจำ	เหมาะในรูป External Merge Sort

12. Bottom-up Merge Sort

Bottom-up Merge Sort: No Recursion



ภาพที่ 12.1 Bottom-up Merge Sort เริ่มจาก run ขนาด 1 แล้วเพิ่มเป็น 2, 4, 8, ...

Bottom-up Merge Sort ไม่ต้องใช้ recursion โดยมองว่าสมาชิกแต่ละตัวคือ sorted run ขนาด 1 จากนั้น Merge run ที่อยู่ติดกันเป็นขนาด 2 ,แล้วขนาด 4 ,8 และเพิ่มเป็นเท่าตัวจนคลุมข้อมูลทั้งหมด

ข้อดี

- หลีกเลียง recursive call stack
- โครงสร้างการทำงานเหมาะกับการรวมและบางระบบที่ต้องการควบคุมหน่วยความจำ
- ยังคงเวลา $\Theta(n \log n)$ และสามารถรักษา stability ได้

แนวคิด Pseudocode

```
width = 1
while width < n
    merge adjacent runs of size width
    width = width * 2
```

13. Merge Sort กับ Linked List

Merge Sort เหมาะกับ Linked List เพราะขั้น Merge สามารถเปรียบเทียบหัวของสองลิสต์แล้วเชื่อม node ที่เลือกต่อท้ายผลลัพธ์ได้ โดยไม่ต้องเลื่อนสมาชิกจำนวนมากเหมือน Array

ขั้นตอนทั่วไป

- 1. หา midpoint ของ linked list (เช่น slow/fast pointers)
- 2. ตัดลิสต์เป็นสองครึ่ง
- 3. เรียก Merge Sort กับแต่ละครึ่ง
- 4. Merge สอง sorted linked lists ด้วยการเชื่อม next pointers

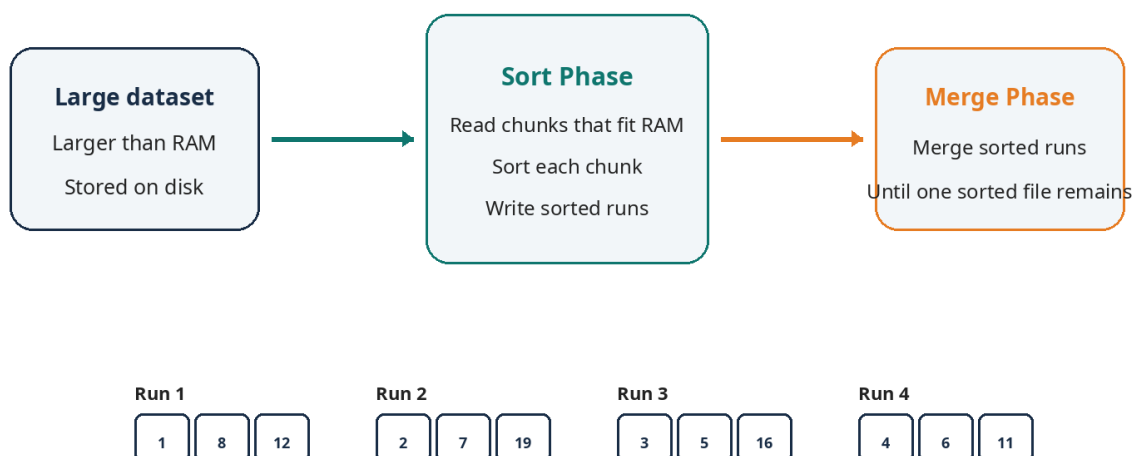
จุดเด่น สำหรับ Linked List ขั้น Merge สามารถทำได้โดย re-link node จึงไม่จำเป็นต้องสร้าง auxiliary array ขนาด n แบบ Array implementation ทั่วไป

ความซับซ้อน

รายการ	ค่า
เวลา	$\Theta(n \log n)$
พื้นที่เพิ่มจากการ Merge	สามารถเป็น $O(1)$ นอกเหนือจาก node เดิม หากเชื่อม node โดยตรง
call stack แบบ recursive	$O(\log n)$

14. External Merge Sort

External Merge Sort Concept



ภาพที่ 14.1 แนวคิด *External Merge Sort* สำหรับข้อมูลที่ใหญ่กว่าหน่วยความจำหลัก

เมื่อข้อมูลมีขนาดใหญ่จนใส่ใน RAM ไม่หมด การเรียงต้องคำนึงถึงการอ่าน-เขียนดิสก์ *External Merge Sort* จะแบ่งกระบวนการเป็นการสร้าง *sorted runs* ที่พอดีกับหน่วยความจำ และการ *Merge runs* เหล่านั้นจนเหลือผลลัพธ์เรียงแล้วชุดเดียว

ภาพรวม

1. อ่านข้อมูลเป็นก้อนที่พอดีกับ RAM
2. เรียงแต่ละก้อนในหน่วยความจำ
3. เขียนก้อนที่เรียงแล้วกลับดิสก์เป็น *sorted run*
4. Merge หลาย *run* โดยอ่านส่วนหน้าเข้ามาทีละน้อย
5. ทำซ้ำจนเหลือไฟล์ผลลัพธ์เรียงแล้ว

แนวคิดนี้เป็นเหตุผลหนึ่งที่ *Merge Sort* มีบทบาทสำคัญในการจัดการข้อมูลขนาดใหญ่และระบบฐานข้อมูล

15. ข้อผิดพลาดที่พบบ่อย

ข้อผิดพลาด	ผลที่เกิด	วิธีแก้
ไม่มี Base Case	Recursion ไม่จบ	หยุดเมื่อ length <= 1
แบ่งแล้วขนาดไม่ลด	อาจเกิด infinite recursion	ตรวจ mid และช่วง index
Merge สองฝั่งที่ยังไม่เรียง	ผลลัพธ์ไม่รับประกันลำดับ	ต้องให้ recursive calls เสร็จก่อน
ลืมคัดลอกสมาชิกที่เหลือ	ข้อมูลบางตัวหาย	มีชั้น append remainder
ใช้ < แทน <= เมื่ออยาก stable	ลำดับ record ที่ key เท่ากันอาจสลับ	เลือก Left เมื่อค่าเท่ากัน
เพิ่ม pointer ผิดฝั่ง	ข้ามข้อมูลหรือวนไม่จบ	เพิ่ม i เมื่อเลือก Left และเพิ่ม j เมื่อเลือก Right

Checklist ตอน Debug

- ทุก call ทำให้ input เล็กลงหรือไม่?
- Base Case return จริงหรือไม่?
- Left และ Right ที่ส่งเข้า merge เรียงแล้วหรือไม่?
- Result มีจำนวนสมาชิกเท่ากับ Left + Right หรือไม่?
- ค่าทุกตัวจาก input ยังอยู่ครบหรือไม่?

16. คำถามที่มักสับสน (FAQ)

ทำไมข้อมูล 1 ตัวจึงถือว่าเรียงแล้ว?

เพราะไม่มีคู่สมาชิกใดที่ขัดลำดับ จึงไม่ต้องทำงานเพิ่ม และใช้เป็น Base Case ได้

Merge กับ Merge Sort ต่างกันอย่างไร?

Merge คือขั้นตอนรวม sorted sequences สองชุด ส่วน Merge Sort คือกระบวนการทั้งหมดที่รวมทั้ง Divide, recursive sorting และ Merge

Merge Sort เรียงจากมากไปน้อยได้ไหม?

ได้ เพียงเปลี่ยนเงื่อนไขใน Merge ให้เลือกค่าที่มากกว่าก่อน และยังสามารถรักษา stability ด้วยกติกา tie ที่เหมาะสม

ทำไม Merge Sort ยังเป็น $n \log n$ เมื่อข้อมูลเรียงแล้ว?

เวอร์ชันมาตรฐานยังแบ่งเป็นทุกระดับและ Merge กลับเหมือนเดิม จึงยังทำงานตามโครงสร้าง $n \log n$ แม้ comparison บางส่วนอาจง่ายขึ้น

Merge Sort เป็น in-place ไหม?

Array implementation ทัวไปไม่เป็น เพราะใช้พื้นที่ช่วย $O(n)$ ส่วน in-place variants มีได้แต่ซับซ้อนกว่า

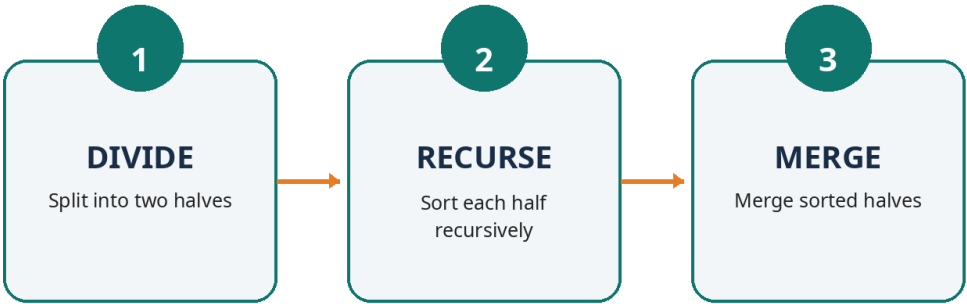
16.1 Top-down vs Bottom-up

หัวข้อ	Top-down	Bottom-up
การควบคุม	Recursion	Loop
เริ่มจาก	Array ใหญ่แล้วแบ่งลง	Run ขนาด 1 แล้วรวมขึ้น
Call stack	$O(\log n)$	ไม่มี recursive call stack
Time	$\Theta(n \log n)$	$\Theta(n \log n)$
Stability	ทำได้	ทำได้
แนวคิดหลัก	Divide ก่อน Merge	Merge runs ขนาดเพิ่มเท่าตัว

ทั้งสองแบบใช้หลักเดียวกันคือ Merge sorted runs แต่ต่างกันที่ลำดับการสร้าง run และวิธีควบคุมโปรแกรม

17. สรุป Merge Sort ในหน้าเดียว

Merge Sort Workflow



Core idea: split smaller -> sort subproblems -> merge in order

ภาพที่ 17.1 แกนหลักของ Merge Sort

สิ่งที่ต้องจำ	คำตอบสั้น
Base Case	length <= 1
Divide	แบ่งเป็นสองส่วนขนาดใกล้เคียงกัน
Recursive step	เรียง Left และ Right
Merge	เลือกค่าด้านหน้าที่น้อยกว่า
Time	$\Theta(n \log n)$
Array extra space	$O(n)$ โดยทั่วไป
Stable	ทำได้ โดยเลือกฝั่งซ้ายก่อนเมื่อ key เท่ากัน
Bottom-up	Merge run ขนาด 1, 2, 4, 8, ...
Linked List	เหมาะ เพราะ Merge ด้วยการเชื่อม node ได้
External sorting	เหมาะเมื่อข้อมูลใหญ่กว่า RAM

ประโยคสรุป Merge Sort ไม่ได้พยายามจัดทั้งหมดในครั้งเดียว แต่ทำให้ปัญหาเล็กจนง่าย แล้วใช้ “Merge ที่ถูกต้อง” ประกอบคำตอบกลับขึ้นมา

18. แบบฝึกหัด

ส่วน A: ความเข้าใจแนวคิด

- 1) Base Case ของ Merge Sort คืออะไร และเพราะเหตุใดจึงหยุดที่จุดนั้น?
- 2) อธิบายความแตกต่างระหว่างขั้น Divide กับขั้น Merge
- 3) เหตุใดขั้น Merge จึงไม่ต้องค้นหาค่าน้อยที่สุดจากทุกตำแหน่งของทั้งสอง Array?
- 4) ถ้าต้องการรักษา stability เมื่อ $\text{left}[i] == \text{right}[j]$ ควรเลือกฝั่งใดก่อน?
- 5) อธิบายด้วยคำของตนเองว่าทำไม Time Complexity จึงเป็น $\Theta(n \log n)$

ส่วน B: Dry Run

- 6) วาดการแบ่งและรวมของ [12, 7, 3, 9, 2, 10]
- 7) Merge [1, 4, 8, 1, 2] กับ [2, 3, 10] พร้อมเขียน Result หลังการเลือกแต่ละครั้ง
- 8) ทำ Dry Run กับ [5, 2, 5, 1] และอธิบายตำแหน่งของ 5 ทั้งสองตัวหากเป็น stable sort

ส่วน C: Code Reading

- 9) หากลบ `result.extend(right[j:])` ออก จะเกิดปัญหาอะไร?
- 10) เปลี่ยน Merge ให้เรียงจากมากไปน้อย โดยอธิบายว่าต้องเปลี่ยนเงื่อนไขใด

18.1 แนวคำตอบ

- 1) หยุดเมื่อมีสมาชิกไม่เกิน 1 ตัว เพราะชุดดังกล่าวเรียงอยู่แล้วและไม่ต้องแบ่งเพิ่ม
- 2) Divide ลดขนาดปัญหา ส่วน Merge ประกอบ sorted subarrays สองชุดเป็น sorted array ที่ใหญ่ขึ้น
- 3) เพราะทั้งสองฝั่งเรียงอยู่แล้ว ค่าที่เล็กที่สุดที่ยังเหลือต้องอยู่ที่ด้านหน้าของฝั่งใดฝั่งหนึ่ง
- 4) เลือก Left ก่อน เพื่อรักษาลำดับเดิมของสมาชิกที่ key เท่ากัน
- 5) มีประมาณ $\log_2 n$ ระดับ และแต่ละระดับมีงาน Merge รวมประมาณ n จึงได้ $n \log n$
- 6) หนึ่งในเส้นทางที่ถูกต้อง: $[12,7,3] / [9,2,10] \rightarrow$ แบ่งต่อจนเดี่ยว $\rightarrow [3,7,12]$ และ $[2,9,10] \rightarrow [2,3,7,9,10,12]$
- 7) Result ตามลำดับ: $[1] [1,2] [1,2,3] [1,2,3,4] [1,2,3,4,8] [1,2,3,4,8,10]$ □ เต็ม 1 2
- 8) ผลคือ $[1,2,5,5]$ และ stable implementation จะคง 5 ตัวที่ปรากฏก่อนให้อยู่ก่อน 5 อีกตัว
- 9) ถ้า Right ยังมีสมาชิกหลัง Leftหมด ข้อมูลส่วนนั้นจะหายจากผลลัพธ์
- 10) เปลี่ยนการเลือกจากค่าน้อยกว่าเป็นค่ามากกว่า เช่น $\text{left}[i] \geq \text{right}[j]$ โดยยังเลือก Left เมื่อค่าเท่ากันถ้าต้องการ stability

19. Glossary สำหรับการเรียนและ RAG

คำศัพท์	ความหมาย
Merge Sort	การเรียงลำดับแบบผสาน; แบ่งข้อมูล เรียงส่วนย่อย และรวมกลับ
Divide	ขั้นแบ่งปัญหาเป็นส่วนย่อย
Conquer	ขั้นแก้ปัญหาย่อย ซึ่งใน Merge Sort ทำผ่าน recursive calls
Merge	รวม sorted sequences สองชุดโดยรักษาลำดับ
Subarray	ส่วนย่อยของ Array
Recursion	การที่ฟังก์ชันเรียกตัวเองกับปัญหาที่เล็กลง
Base Case	เงื่อนไขหยุด recursion; สำหรับ Merge Sort มักเป็น $length \leq 1$
Stable Sort	การเรียงที่รักษาลำดับเดิมของรายการที่มี key เท่ากัน
Auxiliary Space	พื้นที่หน่วยความจำเพิ่มเติมที่ใช้ระหว่างทำงาน
Time Complexity	การเติบโตของเวลาทำงานเมื่อขนาด input เพิ่มขึ้น
Space Complexity	การเติบโตของหน่วยความจำเพิ่มเติมเมื่อ input เพิ่มขึ้น
Top-down Merge Sort	เริ่มจาก Array ใหญ่ แบ่งลงด้วย recursion แล้ว Merge กลับ
Bottom-up Merge Sort	เริ่มจาก run เล็กและ Merge ให้ run ใหญ่ขึ้นด้วย loop
Sorted Run	ช่วงข้อมูลที่เรียงลำดับภายในแล้ว
External Merge Sort	Merge Sort สำหรับข้อมูลที่ใหญ่กว่าหน่วยความจำหลัก โดยใช้ storage ภายนอก

20. เอกสารอ้างอิงและสิทธิการใช้ภาพ

เนื้อหาในเล่มนี้เรียบเรียงใหม่เพื่อการเรียนการสอนและการทำฐานความรู้ โดยใช้แหล่งต่อไปนี้เป็นข้อมูลอ้างอิงเชิงแนวคิดและการตรวจสอบความถูกต้อง

- Miller, B. N., Ranum, D. L., & Yasinovskyy, R. Problem Solving with Algorithms and Data Structures using Python, Runestone Academy. หัวข้อ The Merge Sort. (CC BY-NC-SA 4.0)
- OpenDSA. Mergesort และ OpenDSA License. เนื้อหาโครงการเผยแพร่ภายใต้ MIT License.
- Black, P. E. (ed.). Dictionary of Algorithms and Data Structures, NIST. Entry: merge sort, stable, external sort.
- MIT OpenCourseWare, 6.006 Introduction to Algorithms. Lecture 3: Insertion Sort, Merge Sort.
- Cornell CS 2110. Discussion: Merge Sort for Singly Linked Lists.

ภาพจากแหล่งภายนอก

ภาพที่ 6 .1 สร้างใหม่จากโครงสร้างของ “Merge sort algorithm diagram.svg” โดย VineetKumar, Wikimedia Commons. ต้นฉบับถูกปล่อยเป็น Public Domain. แผนภาพในเล่มนี้ปรับสี ป้ายกำกับ และตัวอักษรภาษาไทยเพื่อการสอน

URL อ้างอิง: https://commons.wikimedia.org/wiki/File:Merge_sort_algorithm_diagram.svg

ภาพอื่นในเล่ม

ภาพที่ 1 . 1 , 4 . 1 , 5 . 1 , 6 . 2 , 9 . 1 , 1 2 . 1 และ 1 4 . 1 จัดทำขึ้นใหม่สำหรับหนังสือเล่มนี้ โดยอาศัยหลักการทางวิชาการที่อธิบายในแหล่งอ้างอิงด้านบน

หมายเหตุสำหรับการใช้เป็น RAG

ข้อมูลสำคัญไม่ได้อยู่ในภาพเพียงอย่างเดียว ทุกภาพมีคำอธิบายข้อความ และคำศัพท์สำคัญถูกเขียนทั้งภาษาไทยและอังกฤษ เพื่อให้ระบบ Retrieval สามารถค้นคืนได้จากข้อความใน PDF

สิ้นสุดเล่ม หากเข้าใจขั้น Merge ได้จริง ส่วนที่เหลือของ Merge Sort จะกลายเป็นเรื่องของการจัดโครงสร้างการแบ่งและการคืนค่าจาก recursion